

CLAVIGER

Rétrospective des interventions humaines ayant orienté la V1

Développement assisté par IA - analyse de l'impact du développeur sur la conception,
l'architecture et la livraison

Objectif du document

Montrer, à partir de l'historique de développement, que l'orientation de Claviger n'a pas été déléguée à une IA. L'assistance IA a produit des propositions, du code et du challenge ; les choix structurants, corrections sémantiques, arbitrages de périmètre et validations ont été répétitivement imposés ou confirmés par l'humain.

*« Je préfère bien poser les bases, histoire qu'on fasse un truc qui, justement, soit modulable
[...] sinon faut refacto, faut recommencer. »*

Extrait utilisateur - phase de cadrage initial

Version de synthèse : V1 - 252 tests automatisés verts + validation fonctionnelle Discord

1. Résumé exécutif

Le point principal ressortant de l'historique est simple : le développeur n'a pas seulement fourni un besoin initial. Il a exercé un rôle actif de product owner technique, de concepteur de domaine et de contrôleur d'architecture tout au long du développement.

Conclusion en une phrase

L'IA a accéléré l'implémentation ; l'humain a régulièrement défini ce qui devait être implémenté, ce qui ne devait pas l'être, comment les responsabilités devaient être séparées, et a corrigé plusieurs modèles qui auraient sinon produit une architecture moins propre ou un comportement métier moins juste.

- Il a refusé une V1 jetable et imposé une architecture modulaire réutilisable dès le départ.
- Il a séparé les besoins métier des détails Discord et pousse vers des services réutilisables par de futurs adaptateurs Web/API/IA.
- Il a imposé des garde-fous opérationnels : synchronisation explicite, absence de mutation silencieuse au démarrage, preflight des rôles, préservation des rôles hors périmètre.
- Il a corrigé le modèle sémantique IA/non-IA et évité plusieurs sur-modélisations de base de données.
- Il a arbitré le périmètre des versions (V1, V1.1, V1.5) pour éviter le feature creep et garder un chemin de livraison réaliste.
- Il a utilisé les tests automatisés et les essais Discord réels comme instruments de décision, pas comme simple vérification finale.

Ce document ne cherche donc pas à attribuer chaque ligne de code à une personne. Il documente plutôt **la responsabilité de direction technique** : formulation des contraintes, choix de modèle, refus de certaines abstractions, arbitrages de sécurité et validation des comportements.

2. Méthode de lecture

La rétrospective a été construite à partir de la chronologie des échanges consacrés à Clavier. Une intervention est retenue lorsqu'elle a au moins un des effets suivants :

- change l'architecture ou la frontière entre composants ;
- corrige un modèle métier ou une interprétation fonctionnelle ;
- réduit un risque de sécurité, de mutation destructive ou de dette technique ;
- évite une duplication ou une abstraction prématurée ;
- modifie le périmètre d'une version ou l'ordre de livraison ;
- fait apparaître, via un test réel, une incohérence non visible dans le code seul.

Important

Les formulations entre guillemets sont réservées aux extraits clairement présents dans l'historique accessible. Les autres fiches sont des synthèses de décisions : elles décrivent le sens de l'intervention sans prétendre restituer mot pour mot la formulation d'origine.

3. Interventions structurantes - chronologie

01

14 août 2026

ARCHITECTURE

Refuser le code jetable et imposer une base modulaire

Situation initiale - Le premier objectif pouvait être traité comme un simple bot d'attribution de rôles, avec une logique directement collée aux commandes Discord.

Intervention humaine - Le développeur rejette explicitement l'idée de « faire du jetable » et demande de poser des bases modulaires, réutilisables et évolutives, quitte à investir davantage au début.

Effet sur le projet - Cette intervention change le projet de nature : les commandes deviennent des points d'entrée sur des services, les responsabilités sont séparées, et la V1 est pensée comme le premier module d'un système plus large plutôt qu'un script isolé.

Trace / indice - Extrait : « Je préfère bien poser les bases [...] modulable [...] sinon faut refacto, faut recommencer. »

02

21 août 2026

CONCEPTION

Séparer les commandes Discord de la logique de gestion des rôles

Situation initiale - Une implémentation fonctionnelle pouvait conserver l'ajout/retrait de rôles dans les handlers de commandes.

Intervention humaine - Le développeur demande que les commandes restent fines et que la gestion des rôles passe par un RoleManager et des services réutilisables. Il demande aussi des tests sur les comportements importants plutôt que des tests mécaniques de fonctions.

Effet sur le projet - La logique devient testable hors Discord, réutilisable par d'autres interfaces et moins couplée à discord.py. Ce choix prépare directement les futurs outils IA et une interface Web.

Trace / indice - Décision cohérente avec son expérience .NET : commande = adaptateur, service = logique partagée.

03

21 août 2026

OPERATIONS

Interdire les mutations silencieuses au démarrage

Situation initiale - Une stratégie classique aurait pu scanner les rôles et mettre à jour la base automatiquement au lancement du bot.

Intervention humaine - Le développeur impose que la découverte et la synchronisation soient déclenchées manuellement par commande : le démarrage observe, mais ne modifie ni Discord ni la base de manière implicite.

Effet sur le projet - Le comportement devient auditable et réversible : scan, diagnostic, puis synchronisation explicite. Une erreur de convention ou de détection ne peut pas reconfigurer silencieusement un serveur au boot.

Trace / indice - Principe retenu : startup sans side effect métier ; sync explicite par administrateur.

04

21 août 2026

SÉCURITÉ

Retenir une défense en profondeur pour les rôles

Situation initiale - Se fier uniquement à la hiérarchie Discord aurait été plus simple : si le bot peut gérer un rôle, il le modifie.

Intervention humaine - Le développeur valide une approche plus stricte : Discord reste la vérité opérationnelle, mais la configuration/policy, les préfixes attendus et la hiérarchie doivent tous être cohérents. En cas d'incohérence, le système refuse plutôt que deviner.

Effet sur le projet - Le bot passe d'une logique « techniquement possible » à une logique « explicitement autorisée ». Cela réduit fortement le risque qu'un changement de serveur fasse sortir Claviger de son domaine de responsabilité.

Trace / indice - *Orientation fail-closed et moindre privilège.*

05

21-24 août 2026

DONNÉES

Privilégier des identités stables et une découverte dynamique

Situation initiale - Les noms de rôles et salons auraient pu être utilisés comme identifiants permanents.

Intervention humaine - Le développeur insiste sur des conventions de nommage utiles à la découverte, tout en orientant le modèle vers des identifiants Discord stables et des noms conservés comme snapshots/configuration humaine.

Effet sur le projet - Le modèle résiste mieux aux renommages et prépare le multi-serveurs. Les noms restent utiles à l'humain et à la policy sans devenir l'unique identité technique.

Trace / indice - *Ce principe sera ensuite réutilisé dans les catalogues et les relations guild + role_id.*

06

24 août 2026

ARCHITECTURE

Penser Discord comme un adaptateur, pas comme le cœur du système

Situation initiale - Le projet restait naturellement centré sur les slash commands Discord.

Intervention humaine - Le développeur reformule l'architecture : les commandes Discord sont une interface parmi d'autres au-dessus des mêmes services. Une future Web API ou des tools IA doivent pouvoir réutiliser les mêmes capacités sans recréer la logique.

Effet sur le projet - Cette décision donne une direction de long terme claire : le cœur métier devient indépendant de l'interface, ce qui réduit le coût des futures intégrations.

Trace / indice - *Vision explicite : commandes Discord aujourd'hui, Web/API/IA demain.*

07

29 août 2026

IA / OUTILS

Maintenir une frontière nette entre Claviger et un futur AI Core

Situation initiale - Une évolution « bot + IA » pouvait conduire à injecter progressivement de la logique d'orchestration IA dans Claviger lui-même.

Intervention humaine - Le développeur retient que les implémentations Discord restent dans Claviger ; un futur AI Core n'aura pas à connaître leurs détails techniques et appellera des tools/services exposés proprement.

Effet sur le projet - Cela évite un monolithe IA/Discord et permet de changer de modèle ou d'agent sans réécrire les capacités Discord. L'IA devient consommatrice d'outils, pas propriétaire du domaine.

Trace / indice - Même principe appliqué à d'autres services personnels : une source fonctionnelle, plusieurs consommateurs.

08

6 septembre 2026

ARCHITECTURE

Unifier Interest et Access sans dupliquer l'orchestration

Situation initiale - Deux catalogues distincts pouvaient naturellement mener à deux commandes de sync, deux commandes « next » et deux pipelines quasi identiques.

Intervention humaine - Le développeur rejette cette duplication et choisit un `/claviger catalog sync` et un `/claviger catalog next` globaux, appuyés sur un registry/coordinator commun.

Effet sur le projet - La duplication est éliminée au bon niveau : les repositories restent spécialisés, tandis que l'orchestration commune devient générique. Cela simplifie l'ajout futur d'autres catalogues.

Trace / indice - Intervention explicite contre des commandes Interest/Access séparées.

09

6 septembre 2026

MODÈLE MÉTIER

Corriger la sémantique de `interest-ia` vers `option-ia`

Situation initiale - Le rôle global indiquant la préférence pour du contenu généré par IA avait été pensé comme un « intérêt IA ».

Intervention humaine - Le développeur distingue deux concepts : aimer/parler de l'IA est un centre d'intérêt ; accepter de voir les variantes générées par IA est une option de présentation. Il demande donc `option-ia`, en réservant potentiellement `interest-ia` à un vrai salon sur la génération IA.

Effet sur le projet - Une ambiguïté métier importante disparaît. Le modèle devient extensible sans collision sémantique entre « sujet d'intérêt » et « préférence d'affichage ».

Trace / indice - Renommage structurel : interest-ia -> option-ia.

10

6 septembre 2026

MODÈLE MÉTIER

Définir correctement la relation no-IA / IA

Situation initiale - Le doublement des rôles adultes pouvait être interprété comme deux catalogues indépendants ou pousser à ajouter des colonnes `is\_ai`, `theme\_key`, etc.

Intervention humaine - Le développeur précise que le `no-ia-\*` est la base du questionnaire ; si l'option IA est activée, la variante `ia-\*` correspondante s'ajoute. Le pairing doit utiliser la clé stable `access\_key`, sans migration de schéma inutile.

Effet sur le projet - Le modèle devient plus simple et plus fidèle au besoin : une catégorie choisie + une préférence globale. La base ne stocke pas une sémantique dérivée qu'on peut calculer par convention.

Trace / indice - Règle centrale : option IA additive, jamais substitutive.

11

6 septembre 2026

DONNÉES

Dissocier ce qui est stocké de ce qui est affiché dans le questionnaire

Situation initiale - Avec les doublons no-IA/IA, il aurait été tentant de ne renseigner les descriptions que pour les lignes visibles dans le formulaire.

Intervention humaine - Le développeur clarifie que le formulaire Discord n'affichera que les descriptions des `access-no-ia`, mais que les entrées IA doivent quand même être correctement peuplées ; une description identique est acceptable.

Effet sur le projet - La base reste complète et cohérente indépendamment de l'UI actuelle. Cette décision évite que le modèle de données soit artificiellement appauvri par une seule présentation.

Trace / indice - Correction explicite pendant le peuplement du catalogue adulte.

12

6 septembre 2026

SÉCURITÉ

Limiter chaque workflow à son vrai périmètre de rôles

Situation initiale - Une logique de réconciliation basée uniquement sur un préfixe aurait pu retirer des rôles que le questionnaire ne présentait plus ou qu'un autre système gèrait.

Intervention humaine - Le développeur retient que Noctis et Membre ne doivent modifier que les rôles explicitement gérés par leur workflow/catalogue ; les rôles administratifs, trusted ou autres restent intacts.

Effet sur le projet - La réconciliation devient idempotente sans devenir destructive. Le bot sait non seulement quoi faire, mais surtout ce qu'il n'a pas le droit de toucher.

Trace / indice - Validé en smoke test : ajout/retrait sans alteration des rôles hors périmètre.

13

22 août -> 7
septembre 2026

UX

Faire des salons un élément du parcours, pas un simple décor

Situation initiale - Les commandes `/membre` et `/noctis` auraient pu être utilisables partout, avec seulement une réponse éphémère.

Intervention humaine - Le développeur impose un parcours : lecture dans Vestibulum puis `/membre` dans Salutationes ; pour l'adulte, lecture dans Lex-Noctis puis `/noctis`. Il confirme ensuite que cette contrainte sert de preuve pratique que l'utilisateur a suivi le chemin prévu.

Effet sur le projet - Le design Discord participe au contrôle fonctionnel. La policy contient les noms de salons et la commande vérifie le contexte au lieu de hardcoder un serveur particulier.

Trace / indice - Extrait recent : « Ouais ca prouve qu'ils ont lu. Je valide. »

14

6 septembre 2026

ROADMAP

Préparer le multi-guild sans sacrifier la V1

Situation initiale - Une généralisation immédiate de toutes les conventions Succumbrae aurait pu retarder fortement la livraison.

Intervention humaine - Le développeur veut une architecture multi-guild, avec policy/config par serveur et un futur paramétrage des noms de commandes, mais accepte de conserver certains termes Succumbrae dans la V1 et de faire l'audit sémantique/renommage global en V1.5.

Effet sur le projet - Ce compromis évite deux écueils : hardcoder définitivement Succumbrae ou sur-généraliser avant d'avoir un produit fonctionnel. La dette est identifiée et versionnée.

Trace / indice - V1 fonctionnelle d'abord ; V1.5 pour la genericité sémantique et les renommages.

15

Tout au long de la
V1

QUALITÉ

Faire des tests et du live Discord un outil de conception

Situation initiale - Une approche « code généré puis testé à la fin » aurait pu laisser passer des erreurs d'intégration propres à Discord.

Intervention humaine - Le développeur impose des jalons courts, demande des commits sur état vert et teste régulièrement les parcours réels : catalogues, modales, ajout/retrait de rôles, comportement de defer, noms de salons.

Effet sur le projet - Les tests ne sont pas seulement une preuve de qualité : ils deviennent une boucle de conception. Plusieurs détails impossibles à garantir par lecture seule ont été corrigés avant déploiement.

Trace / indice - État V1 : 252 tests verts et smoke tests Discord réussis.

16

7 septembre 2026

VALIDATION

Corriger `Salutations` en `Salutationes` à partir du domaine réel

Situation initiale - La policy et les tests utilisaient un nom proche mais incorrect du salon cible.

Intervention humaine - Au premier essai live, le développeur relève immédiatement que le salon réel est `Salutationes` en latin et demande de corriger la configuration plutôt que de contourner la commande.

Effet sur le projet - La source de configuration redevient conforme au serveur réel ; aucun hack n'est ajouté pour accepter deux noms. C'est un exemple simple mais parlant de validation humaine du domaine.

Trace / indice - Intervention issue du premier smoke test `/membre`.

17

7 septembre 2026

PRODUIT

Sortir le welcome flow de la V1 pour éviter le feature creep

Situation initiale - Le message privé d'accueil + fallback public et reporting était le dernier gros morceau fonctionnel envisagé avant déploiement.

Intervention humaine - Après validation live des rôles, le développeur propose explicitement de le reporter en V1.1, juste après le premier déploiement.

Effet sur le projet - La V1 obtient une frontière claire : système de rôles complet et déployé d'abord ; onboarding événementiel ensuite. Le projet gagne une release plus courte, testable et versionnable.

Trace / indice - Extrait : « le message d'accueil [...] on le fasse dans une V1.1 juste après le premier déploiement. »

18

7 septembre 2026

LIVRAISON

Faire du README et de l'audit critique des livrables à part entière

Situation initiale - Une fois le code fonctionnel, il aurait été possible d'enchaîner directement sur le déploiement puis de documenter plus tard.

Intervention humaine - Le développeur remplace le README avant le déploiement, puis décide de consacrer une session distincte à une documentation très critique de l'architecture et des logs avant V1.1, tandis que le déploiement sera traité dans la session Mini-PC qui possède déjà le contexte opérationnel.

Effet sur le projet - Cette séparation reconnaît que documentation, audit et exploitation sont trois activités différentes. Elle conserve le contexte approprié pour chacune et prépare une V1.1 sur une base relue plutôt que seulement fonctionnelle.

Trace / indice - Décision de fin de V1 : README poussé, audit critique séparé, déploiement dans la session serveur.

4. Ce que ces interventions ont changé dans l'architecture finale

Sans ces interventions	Avec les décisions retenues
Bot centré sur des handlers Discord	Services réutilisables + commandes comme adaptateurs
Mutation possible au démarrage	Synchronisation explicite et auditable
Rôles gérés selon ce que Discord permet	Rôles gérés selon policy + catalogue + hiérarchie + preflight
Noms comme identifiants principaux	IDs stables, noms comme snapshots/config humaine
Deux pipelines Interest / Access	Registry/coordinator commun, repositories spécialisés
IA traitée comme un intérêt	`option-ia` séparé de tout futur `interest-ia`
Deux variantes de rôles adultes indépendantes	Base no-IA + variante IA additive par `access_key`
Suppression potentielle par préfixe	Périmètre explicite ; rôles hors questionnaire préservés
Généralisation immédiate ou hardcoding durable	V1 ciblée + dette sémantique planifiée en V1.5
Onboarding ajouté avant toute release	Welcome flow reporté proprement en V1.1

5. Répartition réelle des responsabilités humain / IA

La valeur de cette rétrospective est de montrer une collaboration, pas de nier l'assistance IA. Le projet est plus crédible si la repartition des rôles est explicite.

Assistance IA	Responsabilité humaine observée
Proposer des implémentations et des abstractions.	Accepter, refuser ou recadrer l'abstraction selon le besoin réel.
Générer du code, des tests et des refactorisations.	Définir les frontières métier, les contraintes et le niveau de généricité acceptable.
Analyser les erreurs et suggerer des correctifs.	Exécuter le bot, observer Discord réel et signaler

Assistance IA	Responsabilité humaine observée
	les écarts de comportement ou de configuration.
Proposer des schémas de données.	Corriger la sémantique : `option-ia`, pairing par `access_key`, absence de colonnes inutiles.
Suggérer la suite du projet.	Arbitrer la roadmap : V1, V1.1 onboarding, V1.5 généricité, documentation et déploiement séparés.
Aider à documenter les choix.	Assumer publiquement l'usage de l'IA tout en exigeant que la responsabilité de validation reste humaine.

Lecture professionnelle

Ce que l'historique démontre n'est pas « le développeur a tout codé seul ». Il démontre quelque chose de plus pertinent dans un contexte de développement assisté par IA : il a conservé la responsabilité du modèle mental du système, des arbitrages et du niveau de risque acceptable.

6. Compétences mises en évidence par les interventions

Compétence	Éléments observés	Interventions
Architecture logicielle	Modularité, couches, adaptateurs, services réutilisables, frontières AI Core / Discord	1, 2, 6, 7, 8
Modélisation métier	Distinction intérêt / préférence IA, pairing no-IA/IA, périmètre des workflows	9, 10, 11, 12
Sécurité / permissions	Fail-closed, preflight, moindre privilège, préservation des rôles hors domaine	4, 12
Données / persistance	IDs stables, snapshots de noms, catalogue complet indépendant de l'UI	5, 11
UX / produit	Parcours Vestibulum -> Salutationes, séparation membre / adulte	13
Qualité logicielle	Tests de comportement, commits sur vert, smoke tests Discord	15, 16
Gestion de roadmap	V1 ciblée, onboarding en V1.1, généricité en V1.5	14, 17, 18
Développement assisté par IA	Usage de l'IA comme accélérateur sous arbitrage et validation humaine	Toutes

7. Corrections secondaires mais révélatrices

Toutes les interventions n'ont pas changé une couche entière. Certaines corrections locales montrent cependant la même posture : vérifier la sémantique et ne pas accepter une implémentation simplement parce qu'elle « fonctionne ».

Métadonnées des accès IA

Clarification que les descriptions IA peuvent être remplies même si seule la variante no-IA est affichée dans le formulaire. Évite de coupler la qualité de la base à une UI temporaire.

Terminologie des salons

Correction de `Salutations` vers `Salutationes` lors du smoke test réel. La configuration suit le domaine réel ; pas de compatibilité artificielle ajoutée.

Nom de `Officina`

Clarification que le salon concerne les créations personnelles. Améliore la fidélité des libellés/métadonnées utilisateurs.

Emojis et rendu Discord

Validation humaine des choix visuels, avec correction lorsqu'un emoji n'est pas accepté par le composant natif. Rappelle que l'API et le client Discord réel ont le dernier mot sur l'UX.

Séparation des sessions de travail

Documentation critique et déploiement envoyés vers des sessions dédiées disposant du bon contexte. Réduit les décisions prises avec un contexte partiel et garde une trace de travail plus propre.

8. Ce que ce document permet raisonnablement d'affirmer

Ce qu'il permet d'affirmer

Le développeur a eu un impact continu et substantiel sur l'orientation de Claviger. Il a imposé ou validé des principes d'architecture, corrigé des interprétations métier, borné les permissions, arbitre la roadmap et valide les comportements sur l'environnement réel.

Ce qu'il ne cherche pas à affirmer

Il ne prétend pas que chaque implémentation ou chaque idée technique provient exclusivement du développeur. Le développement est explicitement assisté par IA. La valeur démontrée ici est la capacité à piloter cette assistance : comprendre les conséquences, challenger les propositions et conserver la responsabilité des décisions finales.

9. Conclusion

Sur Claviger, l'intervention humaine n'apparaît pas seulement au début et à la fin du projet. Elle est visible dans la forme même de l'architecture finale.

- Le système est modulaire parce que le choix du « non jetable » a été imposé très tôt.
- Le catalogue est générique parce que la duplication Interest/Access a été refusée.
- Le modèle IA est cohérent parce que la sémantique `interest-ia` / `option-ia` a été corrigée humainement.
- La gestion des rôles est prudente parce que le périmètre et les preflights ont été traités comme des contraintes de sécurité, pas comme des détails.
- La V1 est livrable parce que des fonctionnalités utiles mais non essentielles ont été volontairement repoussées aux versions suivantes.
- La qualité est crédible parce que les tests automatisés ont été complétés par des essais Discord réels et des corrections de domaine.

Mon évaluation : si Claviger était présenté uniquement comme « projet généré avec une IA », cette description manquerait une part essentielle du travail. L'historique montre un développement assisté dans lequel l'humain a joué le rôle de concepteur, arbitre et validateur - et, à plusieurs reprises, a empêché le projet de suivre une direction plus simple à court terme mais moins solide à moyen terme.

Annexe - Index rapide des interventions

#	Période	Intervention
1	14 août 2026	Refuser le code jetable et imposer une base modulaire
2	21 août 2026	Séparer les commandes Discord de la logique de gestion des rôles
3	21 août 2026	Interdire les mutations silencieuses au démarrage
4	21 août 2026	Retenir une défense en profondeur pour les rôles
5	21-24 août 2026	Privilégier des identités stables et une découverte dynamique
6	24 août 2026	Penser Discord comme un adaptateur, pas comme le cœur du système
7	29 août 2026	Maintenir une frontière nette

#	Période	Intervention
		entre Claviger et un futur AI Core
8	6 septembre 2026	Unifier Interest et Access sans dupliquer l'orchestration
9	6 septembre 2026	Corriger la sémantique de `interest-ia` vers `option-ia`
10	6 septembre 2026	Définir correctement la relation no-IA / IA
11	6 septembre 2026	Dissocier ce qui est stocké de ce qui est affiché dans le questionnaire
12	6 septembre 2026	Limiter chaque workflow à son vrai périmètre de rôles
13	22 août -> 7 septembre 2026	Faire des salons un élément du parcours, pas un simple décor
14	6 septembre 2026	Préparer le multi-guild sans sacrifier la V1
15	Tout au long de la V1	Faire des tests et du live Discord un outil de conception
16	7 septembre 2026	Corriger `Salutations` en `Salutations` à partir du domaine réel
17	7 septembre 2026	Sortir le welcome flow de la V1 pour éviter le feature creep
18	7 septembre 2026	Faire du README et de l'audit critique des livrables à part entière

Document de rétrospective produit à partir de l'historique de développement de Claviger. Il s'agit d'une analyse qualitative des décisions et corrections, et non d'un journal exhaustif ligne par ligne de la génération du code.